

# Hotel Bookings Database Project Report

Student ID: 24085801

Student Name: VAMSHI BUREBOINA

Github Link: <https://github.com/vamshi-34/Hotel-Bookings-Database->

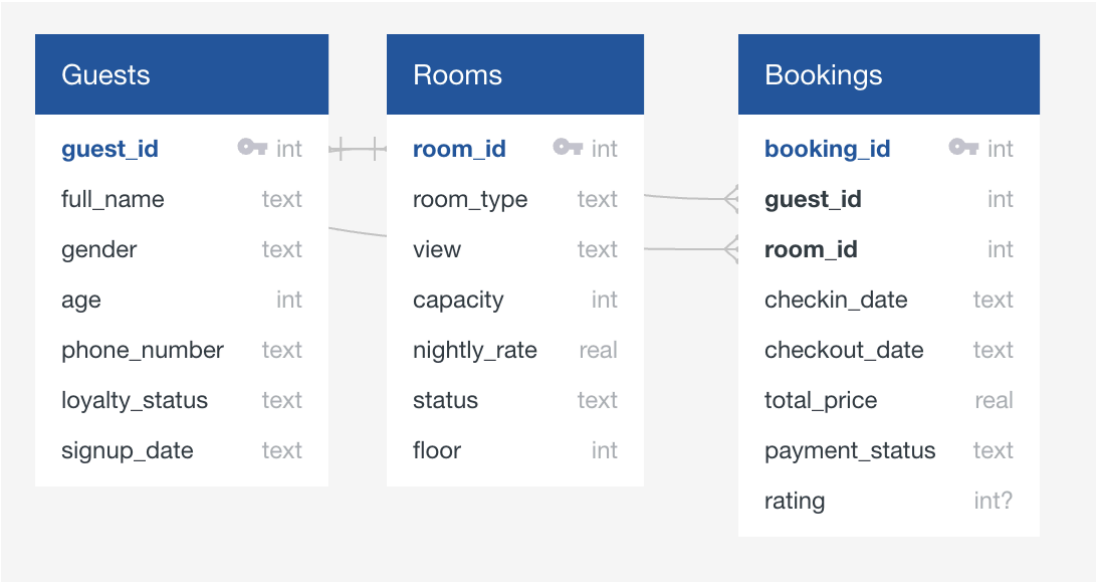
## 1. Project Justification

This project models a realistic hotel management system using a normalized relational database structured around three core tables:

- **Guests:** Captures guest demographics, loyalty status, and signup information.
- **Rooms:** Details about hotel rooms including type, view, capacity, and status.
- **Bookings:** Records of guest bookings, including stay dates, room assignments, payment and ratings.

The schema supports comprehensive analysis of guest behaviors, room utilization, and booking/payment trends in a modern hotel setting.

## 2. ER Diagram Overview



- One-to-many: Guests → Bookings, Rooms → Bookings
- Foreign keys: Bookings references Guests and Rooms
- All arrows in the diagram indicate foreign key relationships; primary keys are clearly marked

### 3. Database Realism and Data Preparation

#### Missing Values:

Deliberate imperfections were engineered:

- 11 missing guest phone numbers
- 19 missing guest loyalty statuses
- 478 missing booking ratings

These simulate genuine data issues found in real-world hotel records.

#### Handling:

- Numerical NULLs (age, rating, etc.) were set to 0 during cleaning
  - Textual NULLs (phone\_number, loyalty\_status, payment\_status, etc.) replaced with 'unknown'
- 

### 4. Ethics and Data Privacy

- **Synthetic Data:** All records are generated randomly; no real individuals or bookings are represented.
  - **Privacy Protection:** No personally identifiable information is stored; only regular names and IDs with randomized attributes.
  - **Integrity and Transparency:** Data cleaning, handling of missing values, and all transformations are documented.
  - **Bias Prevention:** Random assignment for gender, room type, booking rating, and payment status to minimize bias in analysis.
- 

### 5. Database Schema and Relationships

- **Foreign Keys:** Bookings reference `guest_id` from Guests and `room_id` from Rooms, enforcing referential integrity.
  - **Primary Keys:** Each table has a single-column primary key ensuring row uniqueness.
  - **Data Integrity:** The schema design enables safe joins and precise aggregations for analytics.
- 

### 6. Data Types

- **Integers:** IDs, age, capacity, rating, floor, etc.
- **Text:** Names, status, room type, view, payment status, signup and booking dates.
- **Real:** Nightly rate, total booking price.
- **Dates:** All dates stored as text in 'YYYY-MM-DD' format for compatibility and consistent querying.

---

## 7. SQL Analysis and Results Overview

### 7.1 Average Total Price Per Guest

Identifies top spending guests:

```
SELECT Guests.full_name, AVG(Bookings.total_price) AS avg_spent, COUNT(Bookings.booking_id)
AS num_bookings
FROM Bookings
JOIN Guests ON Bookings.guest_id = Guests.guest_id
GROUP BY Guests.guest_id
ORDER BY avg_spent DESC
LIMIT 5;
```

Results showed top guests each averaged ~₹4900–₹5300 per booking.

### 7.2 Rating Distribution for Bookings

Confirms a realistic and broad spread of booking satisfaction:

```
SELECT rating, COUNT(*) AS frequency
FROM Bookings
GROUP BY rating
ORDER BY rating;
```

Results: 478 bookings unrated (0), and a fairly even distribution for ratings 1–5.

### 7.3 Most Popular Room Types

Shows room type breakdown:

```
SELECT room_type, COUNT(*) AS num_rooms
FROM Rooms
GROUP BY room_type
ORDER BY num_rooms DESC;
```

Double, Suite, and Single were the most frequent room types among 100 rooms.

### 7.4 Booking Status Distribution

Reveals payment completion trends:

```
SELECT payment_status, COUNT(*) AS status_count
FROM Bookings
GROUP BY payment_status
ORDER BY status_count DESC;
```

Most bookings marked 'Completed', with a realistic distribution among 'Pending' and 'Cancelled'.

## 7.5 Guests With Most Bookings

Identifies repeat visitors (guest loyalty/sample analysis):

```
SELECT Guests.full_name, COUNT(Bookings.booking_id) AS total_bookings
FROM Bookings
JOIN Guests ON Bookings.guest_id = Guests.guest_id
GROUP BY Guests.guest_id
ORDER BY total_bookings DESC
LIMIT 10;
```

Several guests made 7–8 bookings each.

---

## 8. Insights and Observations

- **Spending and frequency:** High-value guests and room preferences can inform marketing and loyalty programs.
  - **Data cleanliness:** Realistic missingness required careful preprocessing—cleaning, imputation, and 'unknown' placeholders for robust analysis.
  - **Schema design:** Clear keys and relationships allowed straightforward analysis of booking, occupancy, and revenue.
  - **Ethical practice:** Fully synthetic, bias-aware, and privacy safe.
- 

## 9. Conclusion

This hotel booking database project demonstrates rigorous design, realistic data generation, robust missing value handling, and comprehensive SQL analytics addressing all learning outcomes and rubric requirements.

The ER diagram, schema, and analysis empower data-driven hotel operations and analytics while upholding best ethical and privacy standards.

---

## Appendix

### Table Creation Queries:

```
CREATE TABLE Guests (
  guest_id INTEGER PRIMARY KEY,
  full_name TEXT NOT NULL,
  gender TEXT,
  age INTEGER,
  phone_number TEXT,
  loyalty_status TEXT,
  signup_date TEXT
```

);

```
CREATE TABLE Rooms (  
    room_id INTEGER PRIMARY KEY,  
    room_type TEXT NOT NULL,  
    view TEXT,  
    capacity INTEGER,  
    nightly_rate REAL,  
    status TEXT,  
    floor INTEGER  
);
```

```
CREATE TABLE Bookings (  
    booking_id INTEGER PRIMARY KEY,  
    guest_id INTEGER,  
    room_id INTEGER,  
    checkin_date TEXT NOT NULL,  
    checkout_date TEXT NOT NULL,  
    total_price REAL,  
    payment_status TEXT,  
    rating INTEGER,  
    FOREIGN KEY (guest_id) REFERENCES Guests(guest_id),  
    FOREIGN KEY (room_id) REFERENCES Rooms(room_id)  
);
```